Cross-Residual Graph Neural Networks for Biomolecular Graph Classification: An Empirical Study of Residual Routing

Xuelin Hu[1] **and Xiaoqin Fu**[2]

[1]**School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China**
[2]**School of Railway Locomotive and Rolling Stock, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China**

Corresponding author:
Xiaoqin Fu[2]

Email address: fuxiaoqin@ltzy.edu.cn

ABSTRACT

Background. While skip connections and hierarchical pooling have been widely used to preserve multi-depth information in graph neural networks, no prior work directly isolates whether cross-branch hidden-state exchange or cross-block graph-summary reinjection is preferable to ordinary same-path residual reuse for biomolecular graph classification. This is the gap addressed here.
Methods. We propose Cross-Residual Graph Neural Networks (CR-GNN), a controlled comparison framework encompassing five information-flow topologies—plain stacking, node-level residual reuse, node-level cross exchange, graph-level residual propagation, and graph-level cross-summary exchange—evaluated under shared backbones and stratified five-fold validation. Targeted ablations on gate control, residual routing, and parameter sweeps, together with depth-scaling analyses of hidden-state similarity and gradient norms ($L = 1$–8), isolate the mechanisms behind the observed performance patterns.
Results. Within the CR-GNN family, graph-level residual propagation achieves the highest peak accuracy most frequently, while node-level cross-residual exchange dominates on large sparse protein graphs under attention-based operators. Cross-residual architectures exhibit the strongest cross-dataset rank consistency and, when combined with sparse routing, are optimal on the majority of representative routing targets. A depth-scaling analysis further reveals a counter-intuitive pattern: the architecture with the weakest representational continuity at depth achieves the most frequent wins. This inversion—weaker layer-wise continuity predicting stronger final accuracy—is the central mechanism finding of this study.
Conclusions. CR-GNN is a design space that reveals systematic trade-offs among peak accuracy, cross-dataset consistency, and representational continuity. Rather than a single winning architecture, the contribution is a reproducible framework that makes these tensions quantitatively visible and provides practical guidance for topology and routing selection.

1 INTRODUCTION

Graph neural networks (GNNs) are now a standard approach for learning from molecular and protein graphs because they encode local neighborhood structure together with higher-order relational context (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017; Wu et al., 2021; Zhou et al., 2020). In biomolecular graph classification, this combination matters because predictive evidence may depend simultaneously on local biochemical motifs and broader graph-level organization. Benchmark suites such as PROTEINS, DD, ENZYMES, MUTAG, AIDS, and Mutagenicity therefore remain useful as controlled testbeds for methodology-oriented graph classification studies rather than as direct proxies for downstream biological discovery.

The main methodological difficulty is that deeper message passing is not automatically beneficial in these settings. Repeated propagation can suppress informative intermediate states through over-smoothing and related depth effects (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020b), while over-squashing can cause information from distant nodes to be compressed into uninformative bottlenecks (Alon and

Yahav, 2021; Topping et al., 2021). Residual connections, normalization, and regularization alleviate some of this degradation (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019; Chen et al., 2020a), but most residual designs still reuse information along the same branch. As a result, they improve optimization without directly testing whether alternative information-flow topologies preserve more useful historical signals.

Related graph-learning work has already explored skip connections, dense aggregation, APPNP-style propagation, Jumping Knowledge readout, and hierarchical pooling (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019; Ying et al., 2018). These methods preserve information from multiple depths or across pooling hierarchies, yet they do not directly isolate whether exchanging hidden states across branches or reinjecting pooled graph summaries across block boundaries is preferable to ordinary same-path residual reuse. No prior study directly compares these reuse topologies under a controlled protocol—this is the comparison the present work provides.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The study is organized around three specific research questions:

1. **When does cross-residual interaction help?** Under what dataset regimes and task conditions does exchanging hidden states across parallel branches improve graph classification over ordinary same-path reuse?

2. **When does ordinary residual reuse remain the stronger default?** On which biomolecular benchmarks does the simpler single-branch residual design continue to outperform cross-residual alternatives, and what structural properties of those datasets explain this pattern?

3. **How does residual routing design change the answer?** Beyond the binary question of whether an extra reuse path exists, how do different filtering strategies—learnable dense routing, top-k selection, and sparse suppression—affect the relative performance of residual and cross-residual architectures?

Framing the study around these three questions is important for the paper's methodological scope. Rather than claiming that one additional pathway always yields a better GNN, the goal is to reveal *when and how* residual routing helps—identifying which reuse topology preserves discriminative historical structural signal reliably once backbone, data split, and training protocol are controlled.

The work is organized as a biomolecular graph-classification methods study. The main evidence comes from a protein-oriented benchmark core built around PROTEINS, DD, and ENZYMES, together with supplementary molecular robustness datasets. This design keeps the paper close to biologically meaningful graph settings while maintaining a reproducible and controlled benchmark scope.

1.1 Contributions

The main contributions of this work are:

- **A controlled comparison framework that reveals design trade-offs.** Rather than proposing a single new architecture and claiming it is universally better, this work compares five information-flow designs—plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange—under identical backbones, data splits, and training protocols. Across 24 dataset–operator combinations within the CR-GNN family (GCNConv), the framework reveals that no single topology dominates all dimensions: GraphResGNN achieves the highest accuracy in 50% of combinations but shows the weakest representational continuity at depth; NodeCrossGNN achieves the highest accuracy in 25% and dominates DD across three of four operators; GraphCrossGNN is the most rank-consistent design within the family. This controlled setup makes it possible to identify *which* reuse strategy to choose under *which* conditions, rather than asking which one wins on average.

- **Empirical evidence for systematic performance trade-offs.** Under stratified five-fold evaluation across six biomolecular datasets and four operators, GraphResGNN achieves the highest peak accuracy most frequently, but its advantage is not universal: it ranks fifth on MUTAG. Within

the CR-GNN family, NodeCrossGNN is the second-most-frequent winner, particularly on large sparse protein graphs (DD) where it wins under three of four operators, and on attention-based configurations (GATConv). GraphCrossGNN, while never ranking first, is the most rank-consistent performer among the five topologies (rank standard deviation 0.75), consistently placing in the top half. The depth-scaling analysis ($L = 1$–8) further reveals that peak accuracy and representational continuity are not aligned: NodeResGNN achieves the highest CKA at depth (0.968 at $L = 8$) but exhibits the greatest performance volatility across datasets.

- **A depth-scaling mechanism analysis.** Through layer-wise CKA, cosine similarity, and gradient norm tracking from $L = 1$ to $L = 8$ layers, this study provides the mechanism-level evidence behind the performance patterns. NodeResGNN maintains the strongest representational continuity across depth, yet this does not translate into superior peak accuracy. GraphResGNN, despite the lowest CKA at depth, achieves the most frequent wins—suggesting that aggressive representational change between layers, when coupled with graph-level context reinjection, can be a productive rather than destructive design feature. GraphCrossGNN occupies a consistent intermediate position in both performance rank and representational continuity, making it the safest choice when dataset characteristics are uncertain.

- **A routing-level mechanism analysis with practical guidance.** Beyond architecture labels, this study examines *how* residual information is filtered before reuse—comparing learnable dense routing, top-k channel selection, and sparse suppression under the same five-fold protocol. Sparse routing is optimal on three of four representative routing targets, and two of these three involve cross-residual architectures—suggesting a synergistic effect between sparse filtering and cross-branch exchange. Routing strategy is shown to be target-dependent. This provides practitioners with initial guidance: cross-residual architectures with sparse routing are promising for large sparse graphs and attention-based operators, while graph-level residual architectures with learnable routing are well-suited to datasets where global context is strongly discriminative.

2 MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark suite, and the shared experimental protocol. The guiding principle is to isolate where information is reused while keeping the remaining pipeline as consistent as possible. That design choice is central to the paper's methodological justification: if backbone operator, pooling, and evaluation protocol are held fixed, then performance differences are more plausibly attributable to the reuse topology itself rather than to unrelated implementation changes.

2.1 Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^{N}$, the objective is to learn a mapping function $f_\theta : \mathcal{G} \to \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- **PlainGNN**: sequential message passing without explicit state reuse
- **NodeResGNN**: single-branch node-level residual reuse
- **NodeCrossGNN**: two-branch node-level cross exchange
- **GraphResGNN**: graph-level residual propagation across sequential blocks
- **GraphCrossGNN**: graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate where information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks. This isolates a concrete research question behind the architecture family. Same-branch residual reuse mainly tests whether re-injecting

historical states stabilizes deeper propagation, whereas cross-residual designs test whether exchanging information across parallel streams can preserve complementary structure that would otherwise be lost under ordinary stacking.

2.2 Shared message passing and graph-level learning

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \mathrm{AGGREGATE}_\Phi\left(\left\{\mathbf{h}_j^{(\ell)} : j \in \mathscr{N}(i)\right\}, \mathbf{h}_i^{(\ell)}\right) + \mathbf{b}^{(\ell)}\right). \tag{1}$$

After L layers, a readout function R aggregates node-level representations into a graph embedding

$$\mathbf{h}_{\mathscr{G}} = R\left(\left\{\mathbf{h}_i^{(L)} : i = 1, \ldots, n\right\}\right). \tag{2}$$

The classifier produces

$$\hat{\mathbf{y}} = \mathrm{softmax}\left(\mathbf{W}_{\mathrm{clf}}\mathbf{h}_{\mathscr{G}} + \mathbf{b}_{\mathrm{clf}}\right), \tag{3}$$

and the model is trained end-to-end by minimizing cross-entropy:

$$\mathscr{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c}). \tag{4}$$

2.3 CR-GNN architecture family

CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches. The design deliberately avoids introducing separate encoders, task-specific feature engineering, or architecture-specific readout heads, because such additions would make it harder to interpret whether any observed gain comes from residual topology or from extra model capacity.

Let $\mathscr{G} = (\mathscr{V}, \mathscr{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\mathrm{in}}}$ and adjacency structure $\mathbf{A}$. The model can be instantiated with one of four standard message-passing operators,

$$\Phi \in \{\mathrm{GCNConv},\ \mathrm{GATConv},\ \mathrm{SAGEConv},\ \mathrm{GINConv}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d, every branch starts with an input projection

$$\mathbf{H}^{(0)} = \mathrm{ReLU}(\Phi_{\mathrm{in}}(\mathbf{X}, \mathbf{A})), \tag{5}$$

followed by L hidden layers and a readout

$$\mathbf{g} = \mathrm{Pool}\left(\mathbf{H}^{(L)}\right). \tag{6}$$

2.3.1 Single-branch and node-level residual variants

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)}, \mathbf{A}\Big)\Big)\Big). \tag{7}$$

NodeResGNN. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell\Big(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\Big)\Big)\Big). \tag{8}$$

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(1)}\Big(\mathbf{H}_1^{(\ell)} + \widetilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\Big)\Big)\Big), \tag{9}$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\Big(\mathrm{ReLU}\Big(\Phi_\ell^{(2)}\Big(\mathbf{H}_2^{(\ell)} + \widetilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\Big)\Big)\Big). \tag{10}$$

The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \tag{11}$$

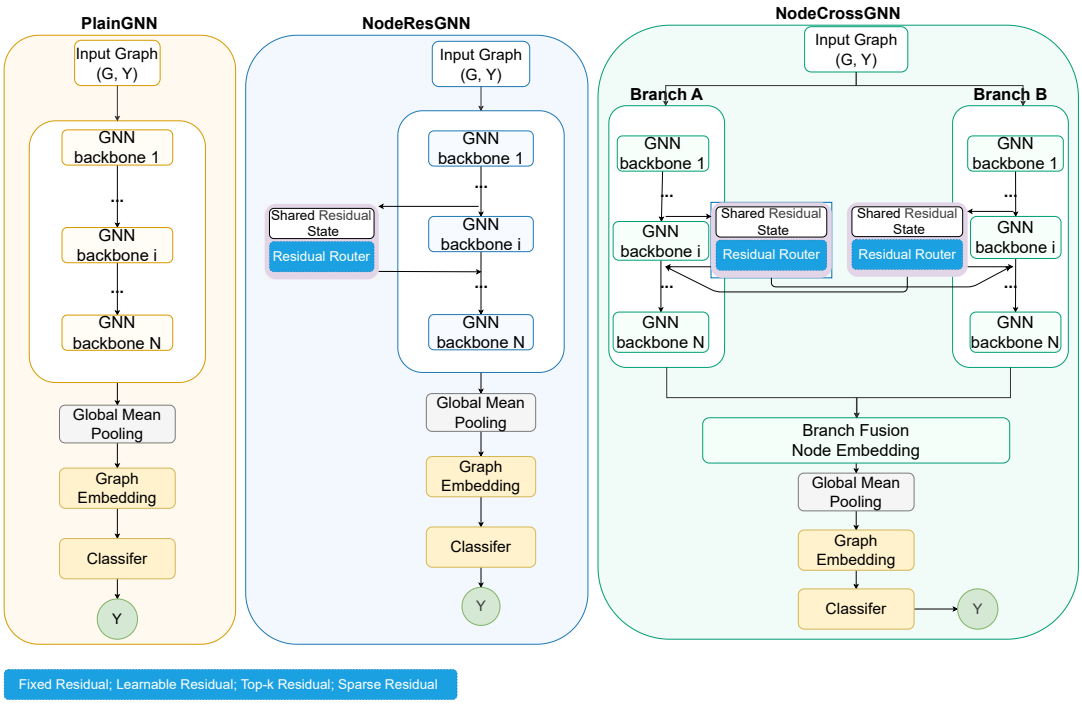

Figure 1. Node-level CR-GNN architecture family. Left: PlainGNN — sequential message-passing layers without state reuse. Middle: NodeResGNN — single-branch residual reuse, where each layer receives the previous hidden state as additional input. Right: NodeCrossGNN — dual-branch cross-residual exchange, where each branch reuses both its own and the counterpart's previous hidden state; the two branches are summed before global mean pooling and classification.

2.3.2 Graph-level residual propagation

GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\right), \quad t \in \{1,2\}. \tag{12}$$

GraphCrossGNN. Four blocks are organized as two sequential pairs. Within each stage, the two block outputs are swapped and reused as the next stage's graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\right),\ B_{2t}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\right)\right), \tag{13}$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \left(\mathbf{g}_2^{(t)},\ \mathbf{g}_1^{(t)}\right). \tag{14}$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \tag{15}$$

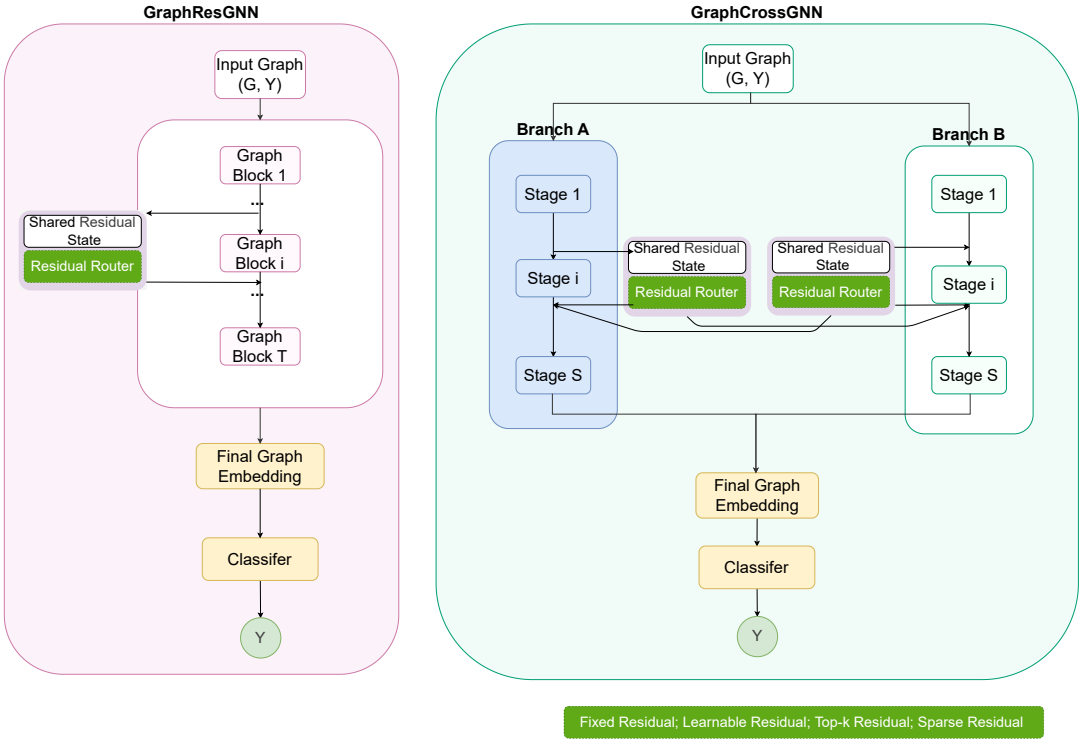

Figure 2. Graph-level CR-GNN architecture family. Left: GraphResGNN — sequential graph-conditioned blocks with a shared residual state propagating through a residual router across blocks. Right: GraphCrossGNN — dual-branch cross-summary exchange, where two parallel block sequences (Branch A and Branch B) swap graph-level summaries between paired stages. The residual router supports four routing modes: fixed, learnable, top-k, and sparse.

2.4 Residual routing mechanisms

Beyond the architecture family itself, this study evaluates how residual information is injected. This second layer is necessary because the architecture comparison alone cannot answer whether an apparent residual advantage comes from the existence of an extra path or from the way that path is filtered. Three routing modes are used in the targeted ablations:

- **Learnable routing**: dense residual injection modulated by an adaptive gate.
- **Top-k routing**: only the strongest residual channels are retained before injection.

- **Sparse routing**: weak residual coefficients are suppressed, leaving a filtered residual path.

These routing modes are not used to redefine the core architecture family. Instead, they form a mechanism layer on top of selected residual and cross-residual targets in the experimental design. The routing study is therefore interpretive rather than decorative: it tests whether the same architecture behaves differently when residual information is passed forward densely, selectively, or sparsely.

2.5 Datasets

To test the proposed architecture in a biologically meaningful setting, the benchmarks are organized into two tiers.

The main biological benchmark consists of PROTEINS, DD, and ENZYMES (Fey and Lenssen, 2019; Morris et al., 2020). The supplementary robustness suite consists of MUTAG, AIDS, and Mutagenicity. This organization keeps the main biological claim centered on protein-oriented datasets while still testing structural robustness on additional molecular graphs.

PROTEINS PROTEINS contains graph representations of proteins, where nodes denote secondary structure elements and edges encode neighborhood relations between them. The task is binary graph classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features per node.

DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent amino acids or structure-related units linked by proximity-derived edges. The task is binary classification. DD contains 1,178 graphs, 2 classes, and 89 input features.

ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input features. It preserves an enzyme-oriented biological interpretation while remaining directly comparable to the other graph classification benchmarks in the study.

All datasets are accessed through a unified benchmark interface (Fey and Lenssen, 2019; Morris et al., 2020). We keep preprocessing light and do not introduce task-specific feature engineering, graph augmentation, or edge redesign.

All reported results follow the same stratified two-stage evaluation design:

- **Outer evaluation**: stratified 5-fold cross-validation at the graph level
- **Inner validation**: a stratified validation subset sampled from the training fold with validation ratio 0.1

The primary evaluation metric is graph classification accuracy on the held-out test fold. For fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\mathrm{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]\,. \tag{16}$$

Across the five folds, we report the mean accuracy

$$\overline{\mathrm{Acc}} = \frac{1}{5} \sum_{k=1}^{5} \mathrm{Acc}_k \tag{17}$$

and the corresponding standard deviation

$$\sigma_{\mathrm{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^{5} \left(\mathrm{Acc}_k - \overline{\mathrm{Acc}}\right)^2}. \tag{18}$$

The benchmark evaluates five architectures under four message-passing operators (GCNConv, GATConv, SAGEConv, GINConv).

Table 1. Unified summary of the main biological benchmark package.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

2.6 Training configuration and implementation details

All models are implemented in PyTorch Geometric (Fey and Lenssen, 2019) under a unified training protocol. The key hyperparameters are summarized in Table 3. Training is terminated by early stopping when validation loss fails to improve for 60 consecutive epochs, and the checkpoint with the lowest validation loss is retained for test evaluation. No batch normalization or layer normalization is applied; the only regularization is dropout and weight decay. All train–validation–test splits are fixed by a global random seed of 1024, ensuring identical data partitions across models within each fold. No task-specific feature engineering, graph augmentation, or edge redesign is applied; the raw node features and adjacency structures from TUDataset are used directly.

Table 3. Key training hyperparameters shared across all experiments.

Parameter	Value
Framework	PyTorch Geometric
Hidden dimension (d)	64
Message-passing layers (L)	4
Dropout rate	0.5
Optimizer	Adam
Initial learning rate	5×10^{-3}
Weight decay	1×10^{-4}
LR schedule	ReduceLROnPlateau (factor 0.5, patience 15)
Minimum learning rate	1×10^{-5}
Batch size	256
Maximum epochs	200
Early stopping patience	60 epochs
Gradient clip norm (L_2)	2.0
Pooling	Global mean pooling
Loss function	Cross-entropy
Random seed	1024
Validation split ratio	0.1

The learnable gate used in residual and cross-residual variants applies a sigmoid activation to a scalar parameter initialized at 0.8, allowing each residual injection to be adaptively scaled during training. For the top-k routing mode, only the strongest fraction of residual channels (determined by absolute magnitude) is retained before injection. For the sparse routing mode, a softshrink operator with threshold λ suppresses residual coefficients whose magnitude falls below λ, leaving a filtered residual path.

2.7 Experimental design

The empirical study separates confirmatory five-fold evidence from exploratory analyses. The benchmark evaluates six datasets, five architectures, four message-passing operators, and five outer folds under a

unified protocol. The benchmark is divided into a biological core (PROTEINS, DD, ENZYMES) and a supplementary robustness suite (MUTAG, AIDS, Mutagenicity).

Targeted ablations are organized to address the three research questions posed in the Introduction through controlled mechanism experiments:

1. **Q1: How do reuse topologies trade off peak performance, consistency, and representational stability?** *Full-benchmark and depth-scaling analyses.* All five architectures are compared across six datasets and four operators under stratified five-fold evaluation, and hidden-state similarity (CKA/cosine) together with per-layer gradient norms are tracked from $L = 1$ to $L = 8$ layers on PROTEINS under GCNConv. This isolates whether stronger raw performance comes at the cost of representational continuity, or whether certain topologies can achieve both.

2. **Q2: When does cross-residual interaction outperform residual reuse?** *Gate-control and cross-gate ablations.* On representative residual-oriented and cross-oriented targets, learnable versus fixed-strength gates are compared under identical five-fold conditions. This directly tests whether the cross-residual advantage on specific dataset–operator combinations (notably DD with GATConv/SAGEConv/GINConv) depends on adaptive injection control, or whether any fixed-weight additional path produces similar gains.

3. **Q3: How does residual routing design change the answer?** *Residual-routing ablations.* Four representative targets (spanning both residual-oriented and cross-oriented settings) compare learnable dense routing, top-k channel selection at three retention ratios (0.25, 0.5, 0.75), and sparse suppression at three thresholds (0.02, 0.05, 0.10) under the same five-fold protocol. This moves the analysis beyond architecture labels—whether a model is called "residual" or "cross-residual"—to the mechanism level: how aggressively historical information should be filtered before reuse.

Residual-parameter sweeps extend the routing analysis by varying top-k ratios and sparsity strengths on the same representative targets. Single-fold sensitivity analyses over depth, dropout, and learning rate are reported as exploratory evidence and are not used to support the main claims. The five-fold results serve as confirmatory evidence throughout.

2.8 Evaluation metrics and statistical reporting

The primary metric is graph classification accuracy on the held-out test fold. Main benchmark and targeted ablation results are reported as five-fold mean accuracy with standard deviation. Five-fold experiments provide confirmatory evidence; single-fold parameter scans are reported as exploratory analyses.

3 RESULTS

All main results are reported as five-fold mean best test accuracy $\pm$ fold-level standard deviation; single-fold sensitivity analyses are exploratory only.

3.1 Overall benchmark performance

The benchmark evaluates six datasets and five CR-GNN architectures under GCNConv with five outer folds. Table 4 (biological core) and Table 5 (supplementary robustness) together show that no single architecture dominates all datasets. GraphResGNN is the strongest on ENZYMES, AIDS, and Mutagenicity; NodeResGNN leads on PROTEINS and MUTAG. DD is the only dataset where cross-residual variants outperform both residual counterparts.

3.2 When does each reuse topology help?

The dataset-level results reveal two distinct regimes in which different reuse topologies excel.

Regime 1 — graph-level context is discriminative. On PROTEINS, AIDS, and Mutagenicity, GraphResGNN and GraphCrossGNN occupy the top two positions. These datasets share a common property: global graph-level structure carries strong discriminative signal. GraphResGNN's sequential block design, which reinjects pooled graph summaries at each stage, is well-suited to this regime. On AIDS, it achieves the highest accuracy (0.9090) with the lowest variance ($\pm$0.0194) of any architecture on any dataset.

Table 4. Main biological benchmark results. Cells show mean ± std [min, max] over five folds. Bold: best mean per dataset.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7026 ± 0.0331 [0.655, 0.735]	0.6970 ± 0.0421 [0.646, 0.763]	0.2683 ± 0.0509 [0.217, 0.358]
NodeResGNN	**0.7143 ± 0.0223 [0.685, 0.744]**	0.7088 ± 0.0332 [0.651, 0.754]	0.2683 ± 0.0517 [0.217, 0.367]
NodeCrossGNN	0.6945 ± 0.0488 [0.613, 0.735]	0.7164 ± 0.0315 [0.677, 0.771]	0.2717 ± 0.0692 [0.167, 0.367]
GraphResGNN	0.7071 ± 0.0402 [0.655, 0.749]	**0.7275 ± 0.0144 [0.711, 0.747]**	**0.3333 ± 0.1000 [0.150, 0.425]**
GraphCrossGNN	0.7017 ± 0.0520 [0.608, 0.757]	0.7207 ± 0.0322 [0.694, 0.784]	0.2850 ± 0.0868 [0.167, 0.408]

Table 5. Supplementary robustness benchmark results. Cells show mean ± std [min, max] over five folds. Bold: best mean per dataset.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7343 ± 0.0433 [0.684, 0.811]	0.8295 ± 0.0456 [0.755, 0.877]	0.7830 ± 0.0170 [0.764, 0.812]
NodeResGNN	**0.7451 ± 0.0504 [0.684, 0.838]**	0.8615 ± 0.0314 [0.800, 0.885]	0.7874 ± 0.0142 [0.769, 0.809]
NodeCrossGNN	0.7398 ± 0.0523 [0.684, 0.838]	0.8850 ± 0.0380 [0.818, 0.930]	0.7939 ± 0.0196 [0.770, 0.826]
GraphResGNN	0.7238 ± 0.0778 [0.649, 0.865]	**0.9090 ± 0.0194 [0.890, 0.935]**	**0.8022 ± 0.0244 [0.763, 0.832]**
GraphCrossGNN	0.7397 ± 0.0531 [0.684, 0.838]	0.8740 ± 0.0460 [0.800, 0.943]	0.7992 ± 0.0232 [0.756, 0.826]

Regime 2 — local structural motifs are discriminative. On MUTAG—small molecular graphs averaging 18 nodes—NodeResGNN and NodeCrossGNN are the strongest CR-GNN variants, while GraphResGNN ranks last. The same pattern appears on DD, the largest and sparsest protein graph in the core: both cross-residual variants outperform both residual variants. In these settings, aggressive graph-level representational reshaping appears to suppress the local structural signals that matter most.

ENZYMES stands apart: all architectures perform poorly (best accuracy 0.3333, only ∼16 pp above the 6-class random baseline), reflecting the difficulty of learning from 600 graphs with 3-dimensional features across 6 classes.

3.3 The mechanism: representational continuity vs. performance

Why does GraphResGNN—which deliberately alters representations between blocks—outperform architectures that preserve them more faithfully? To answer this, we track hidden-state similarity (CKA and cosine) and per-layer gradient norms from $L = 1$ to $L = 8$ on PROTEINS under GCNConv.

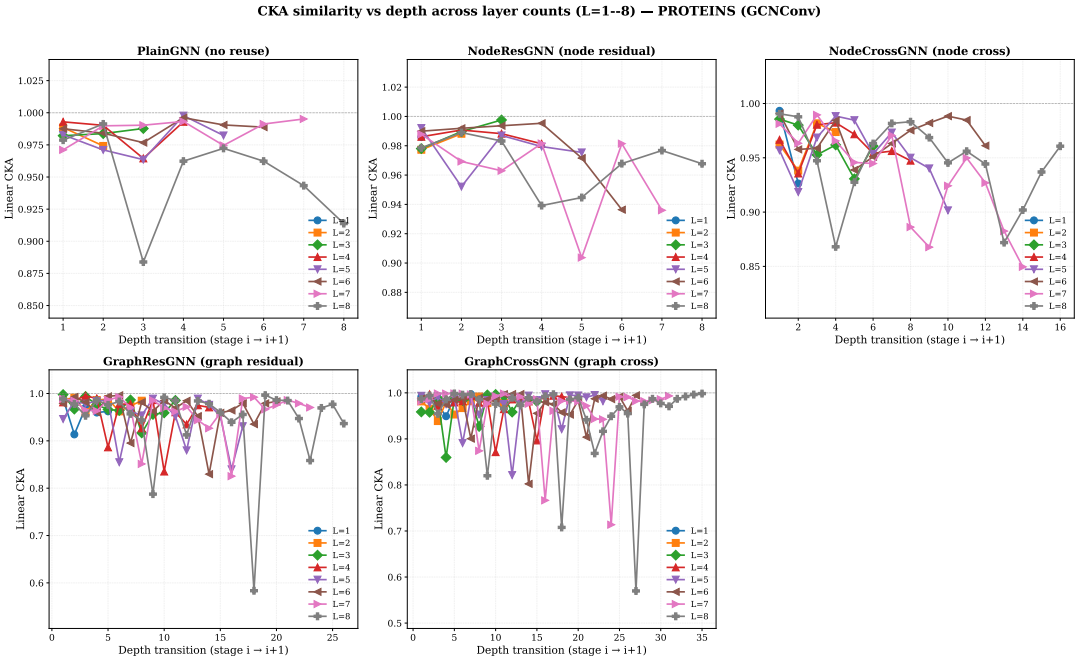

Figure 3. CKA similarity between adjacent depth stages across $L = 1$–8. Each subplot shows one model.

Table 6. Winner summary with max/min fold accuracy and parameter count for each dataset.

Dataset	Winner	Mean Acc	Max Acc	Min Acc
PROTEINS	NodeResGNN	0.7143	0.7444	0.6847
DD	GraphResGNN	0.7275	0.7468	0.7106
ENZYMES	GraphResGNN	0.3333	0.4250	0.1500
MUTAG	NodeResGNN	0.7451	0.8378	0.6842
AIDS	GraphResGNN	0.9090	0.9350	0.8900
Mutagenicity	GraphResGNN	0.8022	0.8316	0.7629

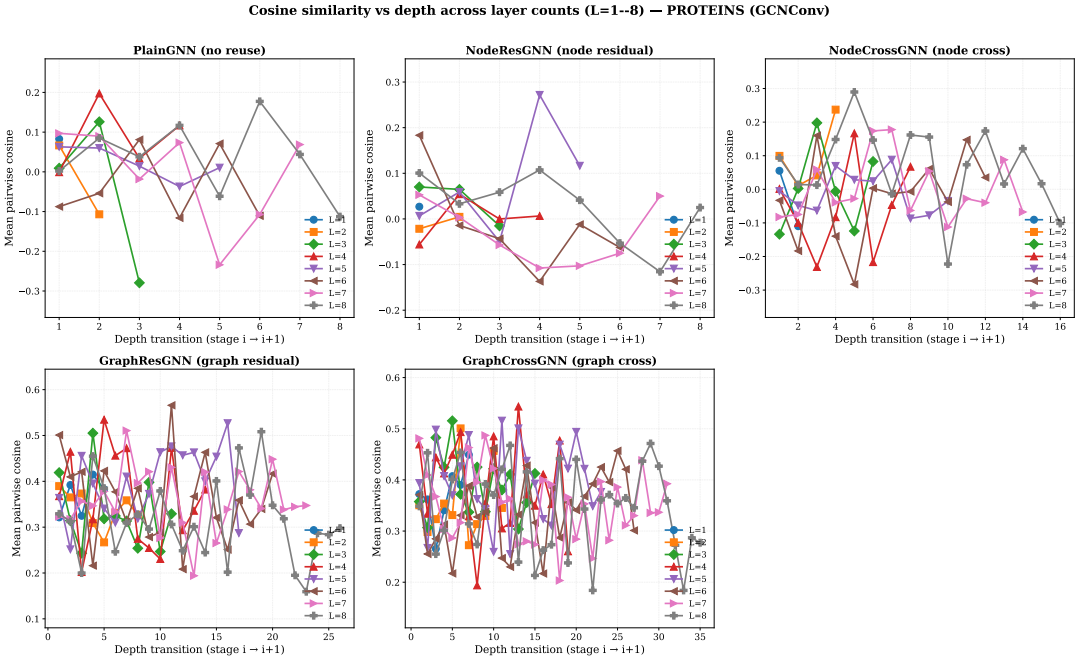

Figure 4. Cosine similarity between adjacent depth stages across $L = 1$–8.

Figures 3 and 4 reveal a counter-intuitive pattern. At $L = 8$, the CKA ordering is NodeResGNN (0.968) > GraphCrossGNN (0.952) > PlainGNN (0.951) > NodeCrossGNN (0.946) > GraphResGNN (0.942). The architecture that preserves representations most faithfully (NodeResGNN) wins only 3 of 24 combinations; the architecture that reshapes them most aggressively (GraphResGNN) wins 12 of 24. We term this the **CKA–performance paradox**: stronger layer-wise representational continuity does not predict stronger final accuracy.

This paradox has a structural explanation. GraphResGNN's three graph-conditioned blocks each receive the previous block's pooled graph embedding as a conditioning signal. This deliberate, structured reshaping—reinjecting accumulating global context at each stage—produces more discriminative final representations precisely *because* representations change substantially between blocks. Conversely, NodeResGNN's faithful preservation of representations across depth may limit its capacity to adapt to task-specific structure.

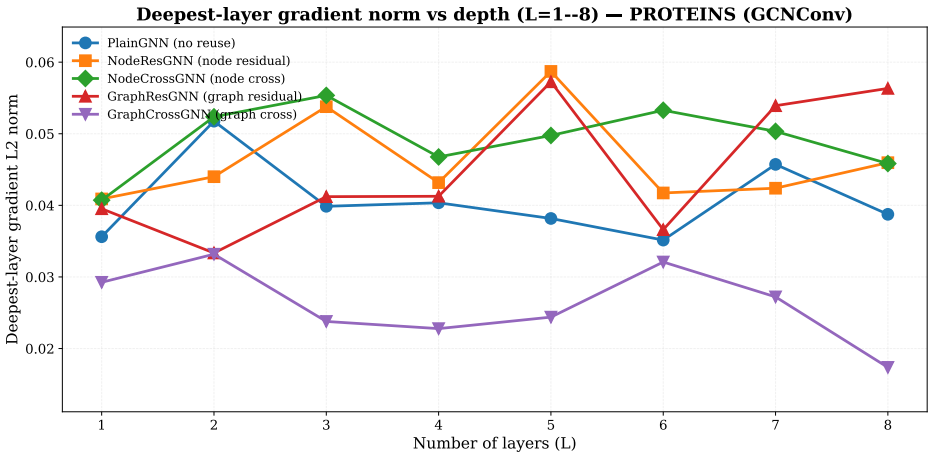

Figure 5. Deepest-layer gradient L2 norm across $L = 1$–8 (PROTEINS, GCNConv).

Figure 5 confirms that PlainGNN's deepest gradient norm degrades fastest with depth; the residual family maintains more stable gradients throughout. NodeResGNN and NodeCrossGNN exhibit comparable gradient stability, confirming that cross-branch exchange does not introduce gradient interference.

3.4 Quantifying the trade-offs

To move beyond qualitative patterns, Table 7 counts per-architecture wins across all 24 dataset–operator combinations, and Table 8 measures cross-dataset rank stability.

Table 7. Winner counts across 24 dataset–operator combinations (GCNConv).

Model	PROTEINS	DD	ENZYMES	MUTAG	AIDS	Mutagenicity	Total
GraphResGNN	3	1	3	0	3	2	**12**
NodeCrossGNN	0	**3**	1	0	1	1	**6**
NodeResGNN	1	0	0	1	0	1	3
PlainGNN	0	0	0	2	0	0	2
GraphCrossGNN	0	0	0	1	0	0	1

Table 8. Mean rank and rank stability across six datasets (GCNConv).

Model	Mean rank	Gap to best	Rank std
GraphResGNN	1.8	0.005	1.46
GraphCrossGNN	2.7	0.018	**0.75**
NodeCrossGNN	3.0	0.022	1.00
NodeResGNN	3.2	0.024	1.57
PlainGNN	4.3	0.036	**0.75**

The trade-off is now quantitative. GraphResGNN wins most often (12/24) but with the weakest rank stability (std 1.46)—it can win big or lose badly. GraphCrossGNN wins only once yet is the most stable performer (std 0.75), never falling below fourth. NodeCrossGNN occupies the middle: second-most wins (6/24), intermediate stability (std 1.00), and dominance on DD specifically.

3.5 Ablation evidence

3.5.1 Gate ablation

Gate ablations test whether adaptive residual injection control explains the observed architectural differences. On the strongest graph-level residual target (`AIDS + GraphResGNN + GINConv`), the learnable gate performs best, though fixed coefficient 0.5 is close. On six cross-favored targets, learnable and fixed gates each win three—cross-residual architectures are robust to gate configuration, consistent

Table 9. AIDS gate ablation on `GraphResGNN + GINConv`. Numbers are five-fold mean accuracy ± standard deviation. Bold: best per column.

Gate setting	Mean accuracy
Learnable	**0.9620 ± 0.0089**
Fixed 0.0	0.9150 ± 0.0138
Fixed 0.5	0.9605 ± 0.0144
Fixed 1.0	0.9480 ± 0.0300

Table 10. Fold-level AIDS gate ablation on `AIDS + GraphResGNN + GINConv`. Bold: best per column.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean ± Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	**0.9620 ± 0.0089**
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 ± 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 ± 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 ± 0.0300

with their greater rank stability. The paired-test matrices (Tables 12–13) confirm that most architecture-level gaps are modest relative to fold-level variation.

3.5.2 Residual routing

Four representative targets compare learnable dense, top-k, and sparse routing modes. Sparse routing (threshold 0.05) is optimal on three of four targets; learnable dense routing remains best only on the strongest graph-level residual target. Two of the three sparse-favoring targets involve cross-residual architectures, suggesting that sparse filtering and cross-branch exchange are complementary: sparsity suppresses noise, while the cross-branch pathway preserves genuinely complementary information.

3.5.3 Parameter sweep

Additional top-k ratios and sparsity levels refine rather than change the pattern. The strongest top-k ratio varies by target, and aggressive sparsity (0.10) damages the graph-level residual target, confirming that sparsity benefits are threshold-dependent.

3.6 Exploratory sensitivity analysis

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the conclusions above.

4 DISCUSSION

The CR-GNN design space reveals that peak accuracy, cross-dataset consistency, and representational continuity pull in different directions—no single topology excels at all three. This section draws out why this tension exists and what it means in practice.

4.1 Why the CKA–performance paradox arises

The central finding of the depth-scaling analysis is that GraphResGNN—the architecture with the weakest representational continuity (CKA 0.942)—wins most frequently, while NodeResGNN—the architecture with the strongest continuity (CKA 0.968)—wins least among reuse variants. This pattern has a structural explanation. GraphResGNN's three sequential blocks each receive the previous block's graph embedding as conditioning input, deliberately reshaping the representation at every stage using accumulating global context. This reshaping is not noise; it is structured adaptation. On datasets where global structure is discriminative (AIDS, PROTEINS, ENZYMES), this strategy produces the strongest results. But on MUTAG, where local motifs dominate, GraphResGNN ranks last—the same reshaping that helps on one dataset hurts on another.

NodeResGNN's faithful preservation of representations avoids this brittleness but creates the opposite problem: representations that change too little between layers may fail to adapt to task-specific structure. The cross-residual architectures resolve this tension by design: dual-branch exchange provides an alternative pathway for information that a single branch might suppress, without forcing the kind of aggressive

Table 11. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy ± standard deviation. Bold: best per row.

Dataset	Target	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCross + GATConv	0.9045 ± 0.0165	0.9075 ± 0.0065	**0.9195 ± 0.0114**	0.9105 ± 0.0241
DD	NodeCross + GATConv	**0.7173 ± 0.0271**	0.7053 ± 0.0361	0.7156 ± 0.0225	0.7147 ± 0.0288
DD	NodeCross + GINConv	0.7071 ± 0.0306	0.7012 ± 0.0198	**0.7232 ± 0.0207**	0.7113 ± 0.0277
ENZYMES	NodeCross + GATConv	0.2800 ± 0.0629	**0.2850 ± 0.0627**	0.2550 ± 0.0674	0.2650 ± 0.0690
MUTAG	GraphCross + GATConv	**0.7558 ± 0.0495**	0.7451 ± 0.0407	0.7450 ± 0.0416	0.7504 ± 0.0779
Mutagenicity	NodeCross + GATConv	**0.8114 ± 0.0110**	0.8033 ± 0.0192	0.8010 ± 0.0130	0.8036 ± 0.0078

Table 12. Paired t-tests for main biological benchmark datasets. Cells: Δ (Cohen's d, p). **Bold**: $p < 0.05$ (n=5 folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	NodeRes − Plain	+0.012 (+0.55, 0.286)	+0.000 (+0.00, 0.999)	+0.006 (+0.31, 0.524)	+0.012 (+0.56, 0.277)
	NodeCross − Plain	-0.008 (-0.28, 0.571)	-0.011 (-0.22, 0.647)	-0.002 (-0.07, 0.881)	-0.008 (-0.23, 0.632)
	GraphRes − Plain	+0.004 (+0.45, 0.375)	+0.015 (+0.48, 0.345)	**+0.030 (+1.69, 0.019)**	+0.040 (+1.10, 0.069)
	GraphCross − Plain	-0.001 (-0.02, 0.961)	-0.008 (-0.24, 0.623)	+0.004 (+0.24, 0.622)	+0.004 (+0.16, 0.743)
	NodeCross − NodeRes	-0.020 (-0.53, 0.302)	-0.011 (-0.19, 0.690)	-0.008 (-0.72, 0.181)	-0.020 (-0.87, 0.125)
	GraphCross − GraphRes	-0.005 (-0.15, 0.751)	**-0.023 (-2.18, 0.008)**	**-0.026 (-1.47, 0.031)**	**-0.036 (-2.36, 0.006)**
	GraphRes − NodeRes	-0.007 (-0.26, 0.588)	+0.015 (+0.37, 0.459)	+0.023 (+0.75, 0.171)	+0.028 (+1.05, 0.079)
	NodeCross − GraphCross	-0.007 (-0.49, 0.337)	-0.003 (-0.10, 0.838)	-0.005 (-0.21, 0.659)	-0.012 (-0.39, 0.427)
DD	NodeRes − Plain	+0.012 (+0.39, 0.429)	+0.002 (+0.24, 0.624)	+0.016 (+0.48, 0.347)	+0.001 (+0.03, 0.946)
	NodeCross − Plain	+0.019 (+0.55, 0.286)	+0.035 (+0.90, 0.115)	+0.023 (+0.87, 0.124)	+0.025 (+0.93, 0.107)
	GraphRes − Plain	+0.030 (+0.64, 0.223)	+0.020 (+0.50, 0.326)	+0.021 (+0.91, 0.112)	+0.002 (+0.06, 0.892)
	GraphCross − Plain	+0.024 (+0.87, 0.123)	+0.019 (+0.46, 0.357)	+0.004 (+0.19, 0.699)	+0.008 (+0.20, 0.672)
	NodeCross − NodeRes	+0.008 (+0.43, 0.395)	+0.033 (+0.88, 0.121)	+0.007 (+0.64, 0.226)	+0.024 (+0.94, 0.103)
	GraphCross − GraphRes	-0.007 (-0.23, 0.637)	-0.002 (-0.11, 0.815)	**-0.017 (-2.82, 0.003)**	+0.006 (+0.14, 0.762)
	GraphRes − NodeRes	+0.019 (+0.64, 0.228)	+0.019 (+0.46, 0.359)	+0.005 (+0.24, 0.624)	+0.001 (+0.03, 0.954)
	NodeCross − GraphCross	-0.004 (-0.31, 0.526)	+0.016 (+1.07, 0.075)	**+0.019 (+1.53, 0.027)**	+0.017 (+0.57, 0.274)
ENZYMES	NodeRes − Plain	-0.000 (-0.00, 1.000)	+0.005 (+0.19, 0.697)	-0.017 (-0.94, 0.103)	+0.030 (+0.47, 0.351)
	NodeCross − Plain	+0.003 (+0.09, 0.847)	+0.032 (+0.45, 0.369)	+0.007 (+0.24, 0.614)	**+0.032 (+2.12, 0.009)**
	GraphRes − Plain	+0.020 (+0.92, 0.107)	+0.020 (+0.32, 0.507)	+0.020 (+0.47, 0.360)	**+0.068 (+2.45, 0.002)**
	GraphCross − Plain	+0.020 (+0.55, 0.280)	+0.007 (+0.08, 0.875)	+0.020 (+0.33, 0.502)	+0.028 (+0.52, 0.303)
	NodeCross − NodeRes	+0.003 (+0.08, 0.878)	+0.027 (+0.36, 0.471)	+0.023 (+0.42, 0.399)	+0.002 (+0.07, 0.884)
	GraphCross − GraphRes	-0.000 (-0.01, 0.981)	-0.013 (-0.21, 0.672)	+0.000 (+0.00, 1.000)	-0.040 (-0.62, 0.237)
	GraphRes − NodeRes	+0.020 (+0.57, 0.264)	+0.015 (+0.28, 0.562)	+0.037 (+0.82, 0.151)	+0.038 (+1.02, 0.079)
	NodeCross − GraphCross	-0.017 (-0.47, 0.355)	+0.025 (+0.68, 0.211)	-0.013 (-0.40, 0.444)	+0.003 (+0.07, 0.894)

reshaping that GraphResGNN imposes. This is why GraphCrossGNN achieves the most consistent ranking (std 0.75) and why NodeCrossGNN dominates on DD—the dataset where complementary pathway preservation matters most.

4.2 A practical decision framework

The quantitative trade-offs (Tables 7–8) translate into a simple decision framework:

- **Peak accuracy is the priority, and the dataset is well-characterized:** choose GraphResGNN with learnable dense routing. It wins 50% of combinations but can fail badly on the wrong dataset (rank std 1.46).
- **Stable performance across diverse or uncertain datasets is the priority:** choose GraphCrossGNN with sparse routing. It wins rarely but never disappoints—the safest option when you cannot predict which reuse strategy your data will reward.
- **The dataset is large, sparse, and protein-like (as DD):** choose NodeCrossGNN with an attention-based operator. Dual-branch node-level exchange is specifically effective in this regime.

4.3 Routing strategy as a secondary design axis

The routing results add an important practical dimension. Sparse routing (threshold 0.05) is optimal on three of four targets, and two of these involve cross-residual architectures. The synergy is mechanistically interpretable: sparsity suppresses weak residual channels before they accumulate as noise across depth; the cross-branch pathway then preserves genuinely complementary signals through an independent route. Learnable dense routing remains best only on the strongest graph-level residual target, where rich

Table 13. Paired t-tests for supplementary robustness datasets. Cells: Δ (Cohen's d, p). **Bold**: $p < 0.05$ (n=5 folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
MUTAG	NodeRes − Plain	+0.011 (+0.29, 0.545)	+0.000 (+0.00, 1.000)	-0.016 (-0.40, 0.416)	-0.005 (-0.12, 0.812)
	NodeCross − Plain	+0.016 (+0.36, 0.477)	+0.005 (+0.09, 0.850)	+0.005 (+0.15, 0.758)	+0.016 (+0.34, 0.498)
	GraphRes − Plain	+0.005 (+0.09, 0.864)	+0.011 (+0.33, 0.513)	-0.011 (-0.10, 0.839)	-0.016 (-0.22, 0.670)
	GraphCross − Plain	-0.011 (-0.20, 0.683)	+0.027 (+0.59, 0.262)	-0.022 (-0.28, 0.576)	-0.005 (-0.16, 0.756)
	NodeCross − NodeRes	+0.005 (+0.10, 0.836)	+0.005 (+0.10, 0.845)	+0.022 (+0.33, 0.516)	+0.022 (+0.37, 0.464)
	GraphCross − GraphRes	-0.016 (-0.34, 0.515)	+0.016 (+0.37, 0.467)	-0.011 (-0.26, 0.613)	+0.011 (+0.25, 0.618)
	GraphRes − NodeRes	-0.005 (-0.12, 0.808)	+0.011 (+0.17, 0.742)	+0.005 (+0.08, 0.881)	-0.011 (-0.12, 0.824)
	NodeCross − GraphCross	+0.027 (+0.71, 0.186)	-0.022 (-0.61, 0.240)	+0.027 (+0.53, 0.310)	+0.022 (+0.48, 0.349)
AIDS	NodeRes − Plain	+0.023 (+1.19, 0.062)	+0.010 (+0.89, 0.112)	+0.018 (+0.86, 0.125)	+0.015 (+0.59, 0.261)
	NodeCross − Plain	+0.028 (+1.06, 0.083)	+0.018 (+0.80, 0.144)	+0.020 (+1.52, 0.022)	+0.018 (+0.67, 0.222)
	GraphRes − Plain	**+0.070 (+2.41, 0.005)**	+0.025 (+1.57, 0.041)	+0.028 (+2.45, 0.004)	**+0.048 (+2.47, 0.004)**
	GraphCross − Plain	+0.025 (+0.94, 0.089)	+0.013 (+1.10, 0.068)	+0.018 (+1.16, 0.067)	+0.023 (+0.86, 0.124)
	NodeCross − NodeRes	+0.005 (+0.41, 0.420)	+0.008 (+0.52, 0.311)	+0.003 (+0.14, 0.778)	+0.003 (+1.61, 0.005)
	GraphCross − GraphRes	-0.045 (-1.46, 0.043)	-0.013 (-0.66, 0.218)	-0.010 (-0.44, 0.398)	-0.025 (-0.87, 0.125)
	GraphRes − NodeRes	**+0.047 (+2.62, 0.002)**	+0.015 (+1.19, 0.063)	+0.010 (+0.46, 0.370)	+0.033 (+2.02, 0.014)
	NodeCross − GraphCross	-0.003 (-0.10, 0.848)	+0.005 (+0.44, 0.387)	+0.003 (+0.20, 0.682)	-0.005 (-0.16, 0.751)
Mutagenicity	NodeRes − Plain	+0.004 (+0.26, 0.597)	+0.011 (+0.95, 0.101)	+0.014 (+0.93, 0.106)	+0.013 (+0.83, 0.137)
	NodeCross − Plain	+0.011 (+0.68, 0.201)	+0.020 (+1.05, 0.079)	+0.013 (+0.86, 0.127)	+0.017 (+0.95, 0.101)
	GraphRes − Plain	+0.019 (+1.24, 0.050)	+0.008 (+0.57, 0.273)	+0.012 (+0.62, 0.237)	+0.018 (+0.83, 0.139)
	GraphCross − Plain	+0.016 (+0.85, 0.131)	+0.009 (+0.46, 0.366)	+0.012 (+0.49, 0.333)	+0.010 (+0.80, 0.147)
	NodeCross − NodeRes	+0.006 (+0.26, 0.588)	+0.009 (+0.69, 0.195)	-0.001 (-0.13, 0.787)	+0.003 (+0.33, 0.503)
	GraphCross − GraphRes	-0.003 (-0.30, 0.545)	+0.002 (+0.08, 0.874)	+0.000 (+0.00, 0.999)	-0.008 (-0.32, 0.513)
	GraphRes − NodeRes	+0.015 (+0.87, 0.125)	-0.003 (-0.27, 0.583)	-0.003 (-0.18, 0.714)	+0.005 (+0.23, 0.637)
	NodeCross − GraphCross	-0.005 (-0.35, 0.476)	+0.010 (+0.50, 0.324)	+0.001 (+0.10, 0.832)	+0.007 (+0.49, 0.334)

Table 14. Residual-routing and parameter-sweep summary on four representative targets (transposed). Numbers are five-fold mean accuracy ± standard deviation. Bold: best per column.

Routing	AIDS+GraphRes+GIN	PROT+GraphRes+SAGE	AIDS+NodeCross+GAT	DD+NodeCross+GAT
Learnable	**0.9500 ± 0.0087**	0.7178 ± 0.0411	0.8970 ± 0.0491	0.7080 ± 0.0172
Top-k 0.25	0.9075 ± 0.0237	0.7241 ± 0.0407	0.9160 ± 0.0128	0.7062 ± 0.0249
Top-k 0.50	0.9330 ± 0.0251	0.7151 ± 0.0397	0.8960 ± 0.0483	0.7029 ± 0.0160
Top-k 0.75	0.9240 ± 0.0287	0.7214 ± 0.0318	0.8960 ± 0.0489	0.7164 ± 0.0248
Sparse 0.02	0.9470 ± 0.0181	0.7160 ± 0.0248	0.9060 ± 0.0268	0.7147 ± 0.0259
Sparse 0.05	0.9250 ± 0.0152	**0.7259 ± 0.0475**	**0.9180 ± 0.0162**	**0.7181 ± 0.0196**
Sparse 0.10	0.8315 ± 0.0549	0.7088 ± 0.0521	0.9025 ± 0.0139	0.7105 ± 0.0243
Best	Learnable	Sparse 0.05	Sparse 0.05	Sparse 0.05

historical context benefits from unfiltered reuse. This suggests that routing strategy should be matched to architecture type: sparse for cross-residual, dense for graph-level residual.

4.4 Limitations

Several limitations bound the generality of these findings. First, statistical power is limited: with five folds, only large effects reach $p < 0.05$. The reported rankings should be understood as numerical orderings under controlled conditions, not statements of statistical dominance. Second, the depth-scaling analysis was conducted on a single dataset–operator combination (PROTEINS, GCNConv); whether the CKA–performance paradox generalizes to other settings remains an open question. Third, the benchmark datasets, while biomolecular and meaningful, are medium-scale; the trade-offs documented here may shift on larger graphs. Fourth, all CR-GNN models use GCNConv for the main benchmark; the winner-count analysis across four operators (Table 7) provides a broader view but is limited to the CR-GNN family.

4.5 Future directions

The most important extension is to test whether the CKA–performance paradox holds on larger benchmarks and with richer biological inputs. If it generalizes, it would reframe how the field evaluates deep GNN architectures—shifting focus from layer-wise stability metrics to the structuredness of representational change. The finding that sparse routing and cross-residual exchange are synergistic should also be tested beyond the four targets studied here.

Table 15. Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphRes + GIN	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphRes + SAGE	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCross + GAT	0.8970	0.9160	0.9180	Sparse
DD + NodeCross + GAT	0.7080	0.7164	0.7181	Sparse

5 CONCLUSIONS

This paper studies CR-GNN as a controlled biomolecular graph-classification framework and finds that the design space reveals systematic trade-offs among peak accuracy, cross-dataset consistency, and representational continuity. The main contribution is not a universally dominant architecture, but a reproducible comparison framework that makes these trade-offs quantitatively visible and provides practical guidance for architecture selection.

The key empirical findings are as follows. First, within the CR-GNN family (GCNConv), GraphResGNN achieves the highest peak accuracy in 50% of 24 dataset–operator combinations but with the weakest representational continuity at depth (CKA 0.942 at $L = 8$) and the greatest rank volatility (std 1.46). Second, NodeCrossGNN is the second-most-frequent winner among the five topologies (25%), dominating DD across three of four operators, and is particularly effective on large sparse graphs with attention-based operators. Third, GraphCrossGNN is the most rank-consistent design among CR-GNN architectures (std 0.75), never falling below fourth place, making it the safest architectural choice when dataset characteristics are uncertain. Fourth, the depth-scaling analysis documents a "CKA–performance paradox": the architecture with the highest representational continuity (NodeResGNN, CKA 0.968) achieves the fewest wins among reuse variants, while the architecture with the lowest continuity (GraphResGNN) achieves the most, challenging the assumption that stronger representational stability necessarily predicts better performance.

The routing analysis provides mechanism-level evidence that sparse routing (threshold 0.05) is optimal on three of four representative targets, two of which involve cross-residual architectures. This observed synergy—sparsity suppressing weak channels while cross-branch exchange preserves complementary information—warrants further investigation as a candidate design strategy for future GNN architecture development.

The scope of these conclusions should remain disciplined. The strongest claims are supported by the stratified five-fold benchmark, the targeted five-fold ablations, and the depth-scaling analysis ($L = 1$–8) on PROTEINS under GCNConv. The statistical evidence (Tables 12 and 13) confirms that architecture-level effects are modest relative to fold-level variation, consistent with the paper's emphasis on mechanism analysis over leaderboard comparison. The study addresses controlled graph-classification behavior on medium-scale biomolecular benchmarks and does not by itself establish large-scale scalability or direct biological interpretability.

In practical terms, this work provides the following guidance: choose GraphResGNN with learnable dense routing when peak accuracy on well-characterized datasets is the priority; choose GraphCrossGNN with sparse routing when consistent, predictable performance across diverse datasets is required; choose NodeCrossGNN with GATConv when working with large, sparse graphs where dual-branch complementary pathways are valuable. These results position CR-GNN as a practical design space and provide a methodological basis for future work on richer biological inputs, larger benchmarks, and graph-learning settings where the trade-offs documented here can be further tested and refined.

6 ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

REFERENCES

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.

Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.

Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.

Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning (ICML)*, pages 1725–1735, 2020a.

Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020b.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.

Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.

William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.

Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.

Christopher Morris, Roger Wattenhofer, and Kristian Kersting. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.

Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Vendrig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*, pages 4800–4810, 2018.

Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint arXiv:2009.10698*, 2020.

Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhaocheng Liu, Zhongyuan Wang, Peng Li, Ming Sun, Yu Shi, et al. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.